

NoSQL Systems — DAS 838

Assignment 1: Data Scaling, Data Integration and Unix Data Processing
Instructor: Vinu E. Venugopal

Team ID: 18, 28

Name	Roll number
Kalpiti	BT2024093
Cheruku Sri Charan Reddy	BT2024143
Arnav Jain	BT2024233
D Sree Teja	BT2024160

Contents

1. [Problem 1 — Scaling Study Using the TPC-H Benchmark](#)
2. [Problem 2 — Incomplete Data Integration and JSON Generation](#)
3. [Problem 3 — Unix Data-Processing Pipeline](#)
4. [Compliance checklist](#)

Problem 1 — Scaling Study Using the TPC-H Benchmark

Experimental setup

Item	Value
Benchmark	Official TPC-H Tools v3.0.1 from tpc.org, archive <code>7C5384AF-10DA-4DCB-B83F-BA2BDE51003F-TPC-H-Tool.zip</code> , SHA-256 <code>97ccb34cd122d78c2e06e2419e50957f934256868b37c02d0b88aefd9d13a84a</code> (included in <code>official_tpch_materials/</code>)
Tool banners (as printed)	TPC-H Population Generator (Version <code>3.0.0 build 0</code>), TPC-H Parameter Substitution (v. <code>3.0.0 build 0</code>): these are the banners of the programs inside the 3.0.1 kit
Database	PostgreSQL 16.15, built from source by <code>setup_postgres_and_dbgen.sh</code>
Machine	Intel Core i9-12900K (16 cores / 24 threads), 62 GB RAM, 931.5 GB NVMe SSD (Crucial CT1000P3SSD8)
OS / compiler	Red Hat Enterprise Linux 8.10 (kernel 4.18), gcc 8.5.0
Scale factors	1, 2, 4, 8, 16, 32 (doubling)
Queries	all 22, generated separately for each SF with the official <code>qgen -r 42 -s <SF></code>
Repetitions	1 warm-up run (excluded) + 3 measured runs of every query at every SF; medians reported
Per-query timeout	30 min at every SF (never reached)

PostgreSQL configuration, identical at every scale factor (database cluster created with `--locale=C --encoding=UTF8`):

```
shared_buffers = 4GB          effective_cache_size = 12GB
work_mem = 128MB             maintenance_work_mem = 1GB
max_parallel_workers_per_gather = 4    max_worker_processes = 8
random_page_cost = 1.1 (SSD)    max_wal_size = 8GB
jit = off                    synchronous_commit = on
```

Indexes (`indexes.sql`): primary keys on all 8 tables, plus indexes on the foreign-key columns and on `l_shipdate` / `o_orderdate`. They are built after loading, identically at every SF.

The environment was unchanged across scale factors. SF1–SF16 ran in one invocation and SF32 in a second, on the same machine, build and configuration. `results/environment.txt` records both invocations. Apart from the date, load average and free memory/disk, the two records are identical: CPU, OS, gcc, PostgreSQL version, every listed setting, tool banners, timeout and repetition protocol. The load average of the 24-CPU machine was 3.74 when the SF1–SF16 invocation started and 0.08 when the SF32 invocation started (both recorded in `environment.txt`).

Procedure (`run_benchmark.sh`, fully automated)

For each scale factor:

1. **Queries.** `generate_queries.sh <SF>` runs the official `qgen -r 42 -s <SF>` for queries 1–22 and saves the exact output (`queries/sf<SF>/raw/`), the PostgreSQL-adapted query (`queries/sf<SF>/<n>.sql`) and SHA-256 of both. The adaptation (`adapt_query.py`) changes only two dialect details. `qgen` (built with `DATABASE=ORACLE`, since the kit has no PostgreSQL option) writes row limits as `where rownum <= N;`, which becomes `LIMIT N` (Q2, Q3, Q10, Q18, Q21). Q1's `interval '...' day (3)` loses the (3) precision, which PostgreSQL does not accept. Scale-dependent parameters come from `qgen`, e.g. Q11's threshold is `0.0001000000` at SF1 and `0.0000031250` at SF32 (= 0.0001/SF). All 7 query sets (SF1–SF32 and the default set) verify against their `SHA256SUMS`. As a cross-check, our 110 raw `qgen` files for SF1–SF16 are byte-identical to the files generated independently by a teammate with the same official kit and seed.
2. **Data.** The official `dbgen -s <SF>` generates the 8 tables. Each `.tbl` file is streamed into `COPY` and deleted after loading. The script **aborts** if a load fails or if the loaded row count differs from the generated line count (`results/row_counts.csv`, all 48 table/SF pairs equal; e.g. `lineitem` 6,001,215 at SF1 and 192,000,551 at SF32).
3. `indexes.sql`, then `VACUUM ANALYZE`. The load time and database size are recorded.
4. **Correctness (SF1).** `validate_answers.sh` runs the 22 queries with `qgen`'s default parameters (`qgen -d`), which the official answer set uses, and compares the results with `dbgen/answers/q<n>.out` using the official tolerances (`check_answers/colprecision.txt`). **22/22 pass.** The kit's own checker `cmpq.pl` also reports 0 unacceptable mismatches for all 22 (`results/answer_validation/tpc_cmpq_summary.txt`). The only non-identical value is Q17 (official 348406.02, ours 348406.05). It is an `avg` column, which the spec allows to differ by up to 1 %; the checker logs it as "OK WITH SPEC".
5. The `EXPLAIN` plan of every query is saved (`results/plans/sf<SF>_q<n>.txt`).
6. **Timing.** All 22 queries run once as a warm-up (run 0), then 3 measured runs. For each run we record the wall-clock time around `psql` (including connection start-up and result transfer) and the status `ok` / `error` / `timeout`.

Reproduce:

```
TPCH_KIT=/path/to/7C5384AF-...-TPC-H-Tool.zip ROOT=$HOME/nosql_a1 ./setup_postgres_and_dbgen.sh
source $HOME/nosql_a1/env.sh
./run_benchmark.sh 1 2 4 8 16      # then, in a second invocation:
./run_benchmark.sh 32
python3 plot_results.py
```

Raw measurements (all in `results/`)

File	Contents
<code>timings.csv</code>	528 rows = 6 SFs × 22 queries × 4 runs (run 0 = warm-up). All 528 are ok
<code>totals.csv</code>	per SF and measured run: raw data size, database size, load time, total of the 22 queries
<code>row_counts.csv</code>	generated vs loaded rows, every table at every SF
<code>scaling_summary.csv</code>	median per query per SF, ratio for every doubling, log-log slope
<code>answer_validation.csv</code> , <code>answer_validation/</code>	SF1 official answer check: our outputs, per-query result, TPC <code>cmpq.pl</code> logs
<code>plans/</code>	<code>EXPLAIN</code> for all 22 queries at all 6 SFs; <code>EXPLAIN (ANALYZE, BUFFERS)</code> at SF32 for Q1, Q5, Q6, Q9, Q10, Q13, Q15, Q16, Q18, Q20, Q21
<code>output/</code>	query results of the last measured run at each SF

File	Contents
environment.txt, tpch_provenance.txt	environment of both invocations; SHA-256 of the kit, dbgen, qgen, dists.dss and the query templates
benchmark_1_16.log, benchmark_32.log	console logs
scaling_plot.png, per_query_plot.png	plots

Totals

SF	Raw data (MB)	DB size (MB)	Load + index (s)	Run 1	Run 2	Run 3	Median total (s)	× previous SF
1	1,049.7	1,704.4	18.4	6.23	6.16	6.31	6.23	—
2	2,114.6	3,397.7	38.1	16.41	15.08	13.84	15.08	2.42
4	4,254.5	6,783.8	83.2	36.48	38.15	37.05	37.05	2.46
8	8,555.8	13,555.6	155.4	69.48	63.63	60.50	63.63	1.72
16	17,220.6	27,102.6	308.4	121.70	123.07	122.74	122.74	1.93
32	34,688.3	54,197.3	636.1	354.27	316.77	328.23	328.23	2.67

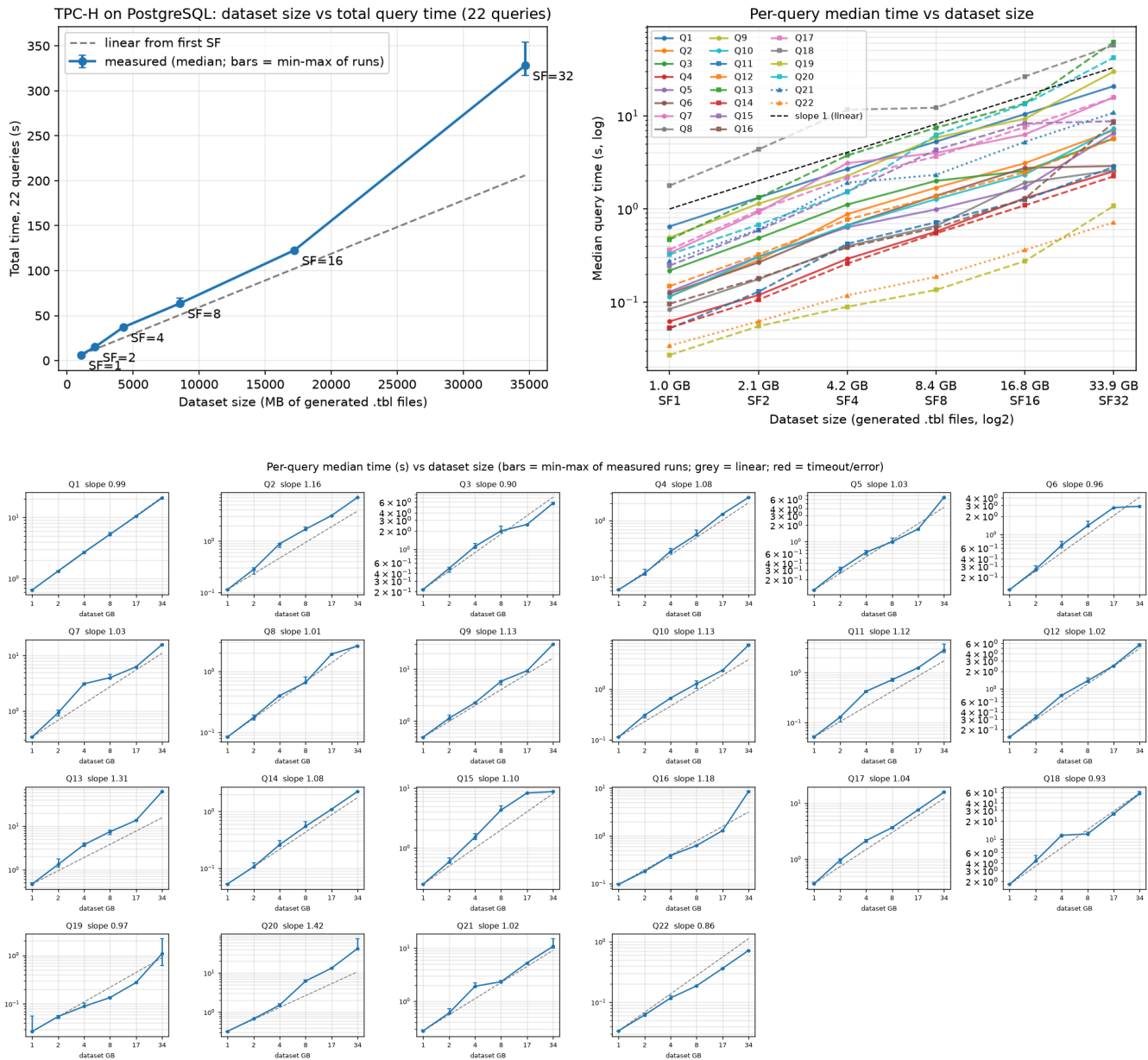
Per query (median seconds, and the ratio at each doubling)

Query	SF1	SF2	SF4	SF8	SF16	SF32	×2/1	×4/2	×8/4	×16/8	×32/16	slope
Q1	0.65	1.32	2.70	5.29	10.42	20.85	2.04	2.03	1.96	1.97	2.00	0.99
Q2	0.12	0.28	0.88	1.69	3.10	6.98	2.45	3.12	1.92	1.83	2.25	1.16
Q3	0.22	0.49	1.11	2.00	2.56	5.73	2.23	2.29	1.80	1.28	2.24	0.90
Q4	0.06	0.12	0.29	0.57	1.30	2.54	1.92	2.45	1.96	2.28	1.95	1.08
Q5	0.13	0.31	0.64	0.99	1.70	6.52	2.39	2.06	1.56	1.71	3.85	1.03
Q6	0.12	0.27	0.67	1.39	2.76	2.90	2.15	2.51	2.08	1.99	1.05	0.96
Q7	0.34	0.92	3.10	4.00	6.30	15.87	2.75	3.36	1.29	1.58	2.52	1.03
Q8	0.08	0.18	0.40	0.66	1.92	2.60	2.10	2.27	1.65	2.91	1.36	1.01
Q9	0.49	1.14	2.26	5.84	9.28	29.86	2.31	1.98	2.59	1.59	3.22	1.13
Q10	0.11	0.30	0.66	1.27	2.34	7.28	2.68	2.18	1.92	1.84	3.11	1.13
Q11	0.05	0.13	0.42	0.72	1.25	2.83	2.48	3.28	1.70	1.74	2.26	1.12
Q12	0.15	0.32	0.77	1.36	2.48	5.67	2.18	2.38	1.77	1.82	2.28	1.02
Q13	0.47	1.32	3.76	7.46	13.55	62.04	2.81	2.85	1.98	1.82	4.58	1.31
Q14	0.05	0.11	0.26	0.55	1.09	2.24	2.00	2.44	2.11	2.00	2.05	1.08
Q15	0.25	0.59	1.53	4.31	8.30	8.75	2.39	2.61	2.81	1.93	1.05	1.10
Q16	0.10	0.18	0.39	0.63	1.29	8.51	1.88	2.15	1.62	2.06	6.61	1.18
Q17	0.37	0.96	2.15	3.66	7.59	15.66	2.63	2.23	1.70	2.08	2.06	1.04
Q18	1.78	4.36	11.64	12.22	26.46	57.21	2.45	2.67	1.05	2.16	2.16	0.93
Q19	0.03	0.06	0.09	0.14	0.28	1.08	2.04	1.62	1.52	2.04	3.92	0.97
Q20	0.33	0.68	1.51	6.21	13.32	42.28	2.09	2.22	4.12	2.14	3.17	1.42
Q21	0.28	0.60	1.91	2.32	5.26	10.82	2.19	3.18	1.22	2.26	2.06	1.02
Q22	0.03	0.06	0.12	0.19	0.36	0.72	1.82	1.90	1.58	1.94	1.98	0.86

A ratio of 2 means the time doubled with the data (linear). **Bold** marks ratios of 3 or more, or 1.4 or less. The slope is fitted to $\log(\text{time})$ vs $\log(\text{dataset size})$ over all six SFs: 1 = linear, above 1 = super-linear.

Performance plots

Dataset size (MB of generated .tbl files) against total and per-query time. The dashed line is linear scaling from SF1; the bars show the min-max of the 3 measured runs.



Analysis

Does query time double when the dataset doubles?

Roughly, but not exactly, and not for every query. From SF1 to SF32 the data grew **33.0×** (1,049.7 MB → 34,688.3 MB) and the total median time grew **52.7×** (6.23 s → 328.23 s), i.e. somewhat faster than linear. The per-doubling ratio of the total stays between 1.72 and 2.46 up to SF16, then rises to **2.67 at SF32**, the largest of all steps. Loading and indexing scale almost exactly linearly (ratios 2.07, 2.19, 1.87, 1.98, 2.06). So

the extra growth comes from query execution, not from the amount of data read. The total also hides three different behaviours, described below. Each one is tied to evidence in the saved plans.

1. Linear queries: scans, filters and aggregation with stable plans

Q1, Q4, Q12, Q14 and Q17 have every ratio between 1.70 and 2.63 and slopes of 0.99–1.08. Their `EXPLAIN` plans have **the same shape at all six scale factors**, so their cost follows the number of rows processed.

- **Q1** is the cleanest case (ratios 1.96–2.04). It is a filter on `l_shipdate` and a hash aggregation into 4 groups. At SF32 the plan is a parallel sequential scan of `lineitem` in 5 processes returning 189 M of the 192 M rows. The aggregate needs 24 kB per process and the final sort 27 kB (`sf32_q1_analyze.txt`). The work is essentially "read every row once", so time doubles with the data.
- **Q12** (join `orders–lineitem`, filter on dates and ship mode, group by ship mode) and **Q14** (join with `part`, one aggregate) behave the same way: filtering and a small aggregation over a scan.
- **Q17** has a correlated subquery (`0.2 * avg(l_quantity)` per part), but its plan is stable, so it stays linear too. A subquery is not super-linear by itself; it depends on how the plan executes it.

Q22 grows a little slower than linear (slope 0.86). It is the fastest query (0.03 s at SF1), and the measured time includes starting `psql`, which does not depend on the data. That fixed part is a larger share of the time for such a short query.

2. Steps caused by the planner changing strategy

Several unusual ratios coincide exactly with a change in the saved plans between two SFs (comparison of `results/plans/sf<A>_q<n>.txt` and `sf<B>_q<n>.txt`):

Query, step	Ratio	Plan at the smaller SF → plan at the larger SF
Q6, SF16→32	1.05	bitmap scan through the <code>l_shipdate</code> index → parallel sequential scan of <code>lineitem</code>
Q15, SF16→32	1.05	serial aggregation of <code>lineitem</code> → parallel (Gather + partial HashAggregate)
Q18, SF4→8	1.05	serial HashAggregate over a <code>lineitem</code> scan → parallel partial HashAggregate
Q21, SF4→8	1.22	different join order and access paths (index lookups, a <code>Memoize</code> node)
Q3, SF8→16	1.28	nested-loop joins with index lookups → parallel hash joins over sequential scans
Q7, SF4→8	1.29	ordinary hash joins → Parallel Hash Join with partial aggregation
Q8, SF16→32	1.36	more of the joins become parallel hash joins
Q20, SF4→8	4.12	hash semi-join → nested-loop semi-join that runs a correlated subquery per row
Q5, SF16→32	3.85	index nested loop into <code>lineitem</code> → parallel sequential scan of <code>lineitem</code> + parallel hash joins
Q10, SF16→32	3.11	index nested loop into <code>lineitem</code> → parallel sequential scan of <code>lineitem</code> + hash join

PostgreSQL chooses plans by estimated cost. As tables grow, index lookups (cheap when few rows qualify) give way to sequential scans, hash joins and parallel workers (cheap when many rows qualify). When the switch pays off, the step is almost flat (Q6, Q15, Q18, Q3). When the new plan does more work per row, the step is steep (Q20, Q5, Q10). **No query is insensitive to dataset size overall.** The flat steps are one-off plan switches, and the next doubling grows again: Q18 goes 1.05 → 2.16, Q7 1.29 → 1.58 → 2.52, Q3 1.28 → 2.24.

3. Super-linear queries, and where performance degrades

Performance degrades most clearly at SF32. The total ratio is the highest there (2.67), and seven queries grow by more than 3× in that single doubling: Q16 6.61×, Q13 4.58×, Q19 3.92×, Q5 3.85×, Q9 3.22×, Q20 3.17×, Q10 3.11×. At SF32 the database (54.2 GB) is close to the machine's RAM (62 GB), and PostgreSQL's

own buffer cache is 4 GB. The SF32 `EXPLAIN (ANALYZE, BUFFERS)` plans show what each of these queries spends its time on:

- **Q13 (customer LEFT JOIN orders, count per customer) — join strategy + random I/O.** At SF16 the plan is a hash right join over a sequential scan of `orders`. At SF32 it is a merge left join that reads `orders` through `idx_orders_custkey`, i.e. in customer order instead of disk order. That index scan alone takes 59.2 s of the 65.2 s. The query makes 18.4 M page reads outside PostgreSQL's buffer cache (`sf32_q13_analyze.txt`).
- **Q16 (part × partsupp, count(DISTINCT ps_supplekey)) — lost parallelism and a sort spilling to disk.** At SF16 it runs a parallel hash join with a parallel merge of sorted results. At SF32 the join is serial, and sorting the joined rows exceeds `work_mem` (128 MB). It becomes an **external merge sort using 193 MB of temporary disk** (`sf32_q16_analyze.txt`).
- **Q9 (six-table join, profit per nation and year) — many index lookups and parallel sorts on disk.** At SF32 the plan looks up `lineitem` through a bitmap index scan **1,390,984 times** and `orders` by primary key **10,433,108 times**. Each of the 5 parallel sorts spills about 135 MB to disk (`sf32_q9_analyze.txt`).
- **Q5 and Q10 (joins driven from orders) — switch to scanning lineitem.** At SF32 both read `lineitem` with a parallel sequential scan and join it against a parallel hash table built from the other join input (Q5: 350 MB; Q10: 936 MB). At SF16 they looked `lineitem` up through an index instead. Q10's 5 parallel sorts also spill about 138 MB each to disk (`sf32_q5_analyze.txt`, `sf32_q10_analyze.txt`).
- **Q20 (nested IN with a correlated sum) — per-row subquery execution.** Since its plan switch at SF8, the correlated subquery (sum of `l_quantity` per part/supplier pair) runs once per candidate `partsupp` row: **786,335 times** at SF32, each a bitmap index lookup into `lineitem` (`sf32_q20_analyze.txt`). Its slope over all SFs is the highest (1.42). Its SF32 runs varied from 41.1 s to 75.7 s, consistent with cache-dependent random I/O.
- **Q18 (group lineitem by order, HAVING sum(l_quantity) > 300) — hash aggregation spilling to disk.** It is linear from SF8 onwards (2.16, 2.16), but at SF32 the aggregation over 192 M rows cannot fit in `work_mem`. The final HashAggregate is split into **133 batches using 4.0 GB of temporary disk**, and each of the 5 parallel partial aggregations writes a further ~1.3 GB (`sf32_q18_analyze.txt`). It is the second-slowest query at SF32 (57.2 s), after Q13.
- **Q19** has the same plan at SF16 and SF32 but grew 3.92×. Its time is tiny (0.28 s → 1.08 s), and its three SF32 runs were 2.21 s, 1.08 s and 0.61 s. With so much variation we do not attribute the growth to a cause.

Relating the observations to database operations

Operation	Observed effect on scaling	Evidence
Filtering / scanning	linear; parallel sequential scans spread the work over 5 processes	Q1, Q12, Q14; Q6 at SF32
Aggregation	linear while the hash table fits in <code>work_mem</code> ; super-linear once it spills to disk in batches	Q1 (4 groups, 24 kB) vs Q18 (133 batches, 4.0 GB on disk at SF32)
Sorting	negligible for small final results; external merge sorts on disk once the sort input exceeds <code>work_mem</code>	Q16 (193 MB), Q9 and Q10 (~135–138 MB per parallel sort) at SF32
Joins	the planner switches between index nested loops, hash joins and merge joins as tables grow. The switch points cause flat steps or sudden jumps	Q3, Q5, Q7, Q10, Q13, Q20, Q21
Correlated subqueries	cost = outer rows × inner lookup; super-linear when the plan runs the subquery per row	Q20 (786,335 executions); Q17 stays linear with a stable plan

Limitations and notes

- **Not an audited TPC-H result.** We run single-stream query timings only: no refresh functions, no throughput test, and we add indexes. This is what the assignment asks for, but it is not a TPC-H benchmark result as defined by the specification.
- **Two invocations.** SF1–SF16 ran first and SF32 in a second invocation later the same day, on the same machine, build and configuration (see above). The server is shared with other users: the load average was 3.74 at the start of the first invocation and 0.08 at the start of the second, and we cannot rule out activity by other users during the runs. The spread between runs is shown in the totals table and as bars in the plots.
- **Some jumps are not explained by plan changes.** At SF2→SF4, Q2 (3.12×), Q11 (3.28×) and Q21 (3.18×) grew fast with an unchanged plan shape, and Q19 at SF32 (above). We report these without a verified cause.
- **EXPLAIN ANALYZE was captured only at SF32** (after the timed runs), for the queries discussed. Plan *shapes* at smaller SFs come from the saved `EXPLAIN` output, which shows the chosen operators but not their actual row counts or memory use.
- **SF32 is the largest scale factor we ran.** The server's disk is shared; about 95 GB was free, and SF32 already needs about 54 GB of database plus up to 34 GB of raw data while loading. SF64 would not fit.
- **Warm-up.** The warm-up run is excluded, but the first measured run is sometimes still slower (SF2: 16.41 s vs 13.84 s in run 3). Medians of 3 runs are reported.

Problem 2 — Incomplete Data Integration and JSON Generation

Files

- `match_customers.py` — the matching/classification program (stdlib only, no dependencies)
- `customer_master.csv`, `customer_incoming.csv` — supplied input data (3000 / 2000 records)
- `classification_results.csv` — one row per incoming record: assigned category + which matching rule fired
- `json_output/` — one JSON document per record with missing, conflicting or irregular information (every `partial_match`, `incomplete_match` and `conflicting` record, plus any other record containing an irregular value)
- `summary_statistics.txt` — record counts per category

Run with (Python 3 standard library only; tested with Python 3.12.10 on Windows 11):

```
python3 match_customers.py
```

The program reads the two supplied CSV files and classifies every incoming record automatically. No record is classified by hand.

Matching strategy

Records are matched against the master dataset using a **priority-ordered chain of identifiers**, from strongest to weakest signal:

1. **customer_id (exact)** — the intended primary key. If the incoming record carries an id that exists in the master file, that master record is the candidate — no ambiguity possible.
2. **email (exact, case-insensitive)** — used only when `customer_id` is absent. Emails are effectively unique per person in this dataset. If more than one master record shares the same email (`email_ambiguous`), the match is deliberately *not* taken — it's classified as **Incomplete Match** rather than guessed.
3. **phone (exact, digits-only comparison)** — same logic as email, one rung down in reliability since phone numbers are more prone to typos and to being shared/reused. Same ambiguity handling.
4. **name + city (both exact, case-insensitive)** — the weakest signal, used only when nothing stronger is available. Names collide far more often than emails or ids, so this rule requires *both* fields to agree and still only fires if it comes back with exactly one candidate.

Priority order matters because a wrong match is worse than no match — e.g. trusting a possibly-mistyped phone number over a correct email would misclassify a genuine new customer as a conflicting existing one.

Classifying once a candidate is (or isn't) found

If exactly one master candidate is found (via whichever rule fires first), every **available** (non-empty) attribute in the incoming record — `name`, `email`, `phone`, `address`, `city` — is compared to the candidate:

- All available attributes agree, none missing → **Complete Match**

- All available attributes agree, but ≥ 1 attribute is empty in the incoming record → **Partial Match**
- Any available attribute disagrees with the master record → **Conflicting Information** (checked before "missing", since a wrong value is worse than an absent one)

If no unique candidate is found:

- The record is empty or near-empty (no id, no name, no contact info at all), or the weak signals matched more than one master record (`*_ambiguous`) → **Incomplete Match** — there genuinely isn't enough reliable information to place the record anywhere.
- The record carries a strong identifier (a `customer_id`, email or phone) or a name+city pair, and none of them matches any master record → **No Match / New Entity**.
- Anything else (e.g. only a name, or only a city) → **Incomplete Match**.

Results on the supplied data

`python3 match_customers.py` classifies all 2000 incoming records:

Category	Records	%	Matched via
Complete Match	400	20.0	customer_id (400)
Partial Match	657	32.9	customer_id (450), name+city (91), email (77), phone (39)
Incomplete Match	193	9.7	no usable candidate
No Match / New Entity	400	20.0	no candidate
Conflicting Information	350	17.5	customer_id (350)

1202 JSON documents are written to `json_output/`: all 1200 partial, incomplete and conflicting records, plus 2 records (one complete match, one new entity) whose only issue is an irregular value. The per-record table `classification_results.csv` lists all 2000 records (row, ids, status, and which rule fired).

Irregular values

Some present values are not in the expected form. They are detected explicitly and reported, not silently discarded:

Irregularity	Incoming rows	Handling
name with leading/trailing spaces and all lower-case (e.g. " chaaya verma ")	39	matched after trimming and case-insensitive comparison; JSON <code>irregular_information</code> keeps the raw value
city with a trailing space (e.g. "Khora ")	2	same

(The master file has 13 cities with surrounding spaces; comparison trims both sides.) Normalisation does not change the category, so a record whose only issue is an irregular value keeps its category (here one complete match and one new entity) and still gets a JSON document listing the irregularity.

Duplicate and empty incoming rows

- **145 customer ids occur 2 or 3 times** in the incoming file (297 rows). Each row is classified independently against the master file, so rows with the same id can get different categories, e.g. `c02666` is conflicting in row 27, a complete match in row 284 and a partial match in row 599. We

report each row as it is and do not merge incoming rows with each other: the task is to classify every incoming record against the master dataset.

- **62 rows are completely empty** (all six fields blank). They are Incomplete Match (see below).

Evidence behind the rule order (measured on the master file)

- **email, phone and name+city are unique in the master file** (0 duplicate emails, 0 duplicate phones, 0 duplicate name+city pairs), so a hit on any of them identifies exactly one customer. The ambiguity branches in the code therefore never fire on this data, but they keep the program safe on other inputs.
- **Name alone is not unique**: 23 names appear more than once in the master file, and 8 of the 400 new-entity records (ids starting with N, whose email and phone match nobody in the master file) share a name with an existing master customer. That is why a record that has *only* a name is Incomplete rather than matched: 104 incoming records have only a name (100 of those names happen to be unique in the master file, 4 are not) and 27 have only a city.
- **62 incoming rows are completely empty**, so they are Incomplete.
- **Incoming ids are consistent**: all 1200 incoming rows with a C... id refer to an id that exists in the master file and none of the 400 N... ids do, so `customer_id` is safe as the first rule.
- **Conflicts are real value disagreements, not formatting noise**: for all 350 conflicting records, the incoming and master values still differ after removing case, spaces and punctuation (e.g. email `ekalinga.narayanan48812@example.in` vs `ikshita.swamy1165@example.in`). 167 records conflict on one field and 183 on two. By field: phone 124, name 108, address 104, city 101, email 96. So exact (case-insensitive) comparison is enough here, and no fuzzy matching was needed.

Why JSON for records with missing, conflicting or irregular information

A flat CSV row can't represent "this field is missing" vs. "this field disagrees with the master record" vs. "this field simply wasn't compared" without inventing sentinel values that get confused with real data. JSON lets each record carry:

- `available_information` — a sub-object of only the fields that *are* present and (if matched) agree,
- `missing_information` — a list of which fields were absent,
- `conflicting_information` — for Conflicting records, both the incoming and master values side by side, so a human reviewer can see exactly what disagreed and decide how to resolve it,
- `irregular_information` — for values present in a non-standard form, the raw value exactly as received and the problem found,

as independently-sized, self-describing pieces — instead of forcing every record into a fixed set of CSV columns where "missing" and "not applicable" look identical.

Assumptions / limitations

- Irregularity detection covers what occurs in this data: surrounding whitespace, and names written entirely in lower case. Emails, phones and ids were also checked: every phone is 10 digits, every email has a single @ and a domain, and every id matches `C#####` or `N#####`. Other forms of irregularity (e.g. misspellings) are not detected.
- Matching is deterministic and rule-based, not a fuzzy/probabilistic matcher (no edit-distance on names, no address normalization) — a misspelled name that doesn't share an id/email/phone with its true master record will be treated as a new entity rather than a partial match. This is a conscious

precision-over-recall choice: we'd rather create a duplicate "new" customer than silently merge two different people.

- Phone comparison strips non-digit characters (spaces, dashes, +91 prefixes would need extra normalization not present in this dataset) but does not attempt cross-country-code equivalence.
- "Conflicting" is checked before "missing" — a record with one wrong field and one missing field is classified Conflicting, not Partial, since the disagreement is the more serious issue to flag.
- A completely blank incoming row (all fields empty, including `customer_id`) is Incomplete Match, not New Entity, since there is nothing to distinguish it as a new customer either.

Difficulties encountered

- **Deciding what "reliably matched" means without a ground truth.** There is no label telling us which incoming row belongs to which customer, so the rule order had to come from properties of the data itself (uniqueness of each attribute, shown above) rather than from accuracy against known answers.
- **Name-only records.** 100 name-only records have a name that appears once in the master file, so matching on name would have been easy. We still did not do it, because 8 of the new customers share names with existing ones. That shows a unique name in *this* master file does not prove it is the same person.
- **Partial vs conflicting when both apply.** A record with one missing field and one disagreeing field fits both definitions. We chose Conflicting, because the category must be unique and the conflict is what needs a human to review it.
- **Rows with no information at all.** An empty row technically "does not correspond to any master record", which reads like New Entity. We classify it as Incomplete, because creating a customer with no attributes would be worse than flagging it.

Problem 3 — Unix Data-Processing Pipeline

Files

- `pipeline.sh`: the complete pipeline
- `scalability_test.sh`: generates ~100 / 250 / 500 / 1000 MB datasets and times the pipeline 3× on each
- `stage_timing.sh`: measures how much time each stage adds (uses `STOP_AFTER`, so it times the real pipeline code)
- `scalability_results.tsv`, `stage_timings.tsv`: measurements for the final pipeline
- `*_before_date_cache.tsv`: the same measurements before the optimisation described below
- `outputs/out_<records>.tsv`: pipeline output for each generated dataset
- `evidence/`: saved outputs of the tests described below — sample and malformed-sample outputs, their rejected rows and stderr, the 200,000-row generated `--malformed` test (output, rejection reasons, first 20 rejected rows), and `reference_check_result.txt`
- `reference_check.py`: independent Python implementation of the same query, used only to verify the pipeline's results (not part of the solution)
- `transactions.tsv`, `transactions_malformed.tsv`, `generate_transactions`, `generate_transactions_README.txt`: supplied files (byte-identical)

Run:

```
./pipeline.sh transactions.tsv                # prints the result
./pipeline.sh transactions_malformed.tsv out.tsv  # bad rows ->
transactions_malformed.malformed_rows.tsv
./scalability_test.sh data/                    # on Windows: GEN="python
./generate_transactions" ./scalability_test.sh data/
./stage_timing.sh data/
```

The pipeline takes any input file name and any number of records. It finds columns by name from the header, so both the generator's order (`transaction_id date category quantity price`) and the sample file's order (`customer_id product_id category quantity price date`) work.

Equivalent SQL

```
SELECT category, COUNT(*) AS transactions, SUM(quantity * price) AS revenue
FROM transactions
WHERE date >= '2026-01-01' AND quantity > 2
GROUP BY category
HAVING SUM(quantity * price) > 100000
ORDER BY revenue DESC
LIMIT 10;
```

Processing stages and SQL → Unix mapping

```
tail -n +2 | awk (validate + WHERE + project) | sort -k1,1 | awk (GROUP BY + HAVING) | sort -k3,3gr  
| head -n 10
```

Stage	Unix command	SQL operation
0	head -n 1 + tr/grep -nx find the column numbers from the header	schema (FROM transactions)
1	tail -n +2: skip the header, stream rows	table scan
2	awk: reject malformed rows into a side file	(data cleaning, not in the SQL)
3	same awk: \$date >= "2026-01-01" && \$qty > 2	WHERE (selection)
4	same awk: print only category, quantity, price	projection
5	sort -t\$'\t' -k1,1 (external merge sort, LC_ALL=C)	brings each GROUP BY key together
6	awk: count and sum qty*price while the key stays the same, emit at each key change	GROUP BY category, COUNT(*), SUM(quantity*price)
7	same awk: emit a group only if revenue > 100000	HAVING
8	sort -t\$'\t' -k3,3gr	ORDER BY revenue DESC
9	head -n 10	LIMIT 10
10	{ printf header; cat; }	output column names

Memory use: every stage reads its input line by line. The aggregation `awk` keeps only the current group's count and sum, not a table of all groups, because `sort` has already put equal categories next to each other. `sort` itself spills to temporary files beyond its 256 MB buffer (`-S 256M`). So no stage holds the whole dataset in memory. Revenue is summed from the original `quantity` and `price` fields and printed with `%.2f`. An earlier version printed large sums in scientific notation (`7.2e+08`), which `sort` then compared as 7.2 and ordered wrongly. It was fixed and checked against an independent Python implementation.

Sample input and output

`transactions.tsv` (supplied sample, complete file, tab-separated):

customer_id	product_id	category	quantity	price	date
101	P01	Electronics	3	12000	2026-01-05
102	P02	Grocery	5	800	2026-01-07
103	P03	Electronics	2	15000	2026-01-10
104	P04	Furniture	4	30000	2026-01-15
105	P05	Grocery	8	1200	2026-02-01
106	P06	Clothing	3	4500	2026-02-03
107	P07	Electronics	6	9000	2026-02-11
108	P08	Furniture	2	50000	2026-02-18
109	P09	Clothing	5	7000	2026-03-02
110	P10	Electronics	4	22000	2026-03-12
111	P11	Grocery	10	1500	2026-03-20
112	P12	Furniture	3	42000	2026-04-01
113	P13	Clothing	2	12000	2026-04-07
114	P14	Electronics	7	11000	2026-04-15
115	P15	Grocery	4	3500	2026-05-03
116	P16	Furniture	5	18000	2026-05-11
117	P17	Electronics	3	27000	2026-05-19
118	P18	Clothing	6	8000	2026-06-01
119	P19	Grocery	2	6000	2026-06-12
120	P20	Furniture	4	25000	2026-06-20

`./pipeline.sh transactions.tsv (stdout):`

category	transactions	revenue
Furniture	4	436000.00
Electronics	5	336000.00

and on stderr: Malformed rows excluded: 0 (see `transactions.malformed_rows.tsv`). Grocery (4 qualifying rows, revenue 42,600) and Clothing (3 rows, 96,500) are removed by `HAVING > 100000`. Both results were checked with an independent Python calculation.

Output for the 1 GB generated file (`outputs/out_25000000.tsv`):

category	transactions	revenue
Electronics	947613	13893935397.29
Furniture	473648	13771332023.54
Home	827826	7190100488.38
Appliances	354066	7188134284.84
Jewelry	237077	6854749898.11
Automotive	355427	6183090146.55
Outdoor	354608	5113672227.36
Sports	592870	4119927717.47
Office	473778	4109592713.89
Clothing	830069	3867625797.67

Malformed-record handling

The validation `awk` checks every row, in this order, and on the first failure writes `reason<TAB>original row` to `<input>.malformed_rows.tsv` and skips the row (next). The pipeline keeps running (set `-e` is deliberately not used).

Check	Reason written
number of fields differs from the header (missing or extra field)	field_count_<n>
empty category	empty_category
quantity is not a non-negative integer ($^{\wedge}[0-9]+\$$)	bad_quantity
price is not numeric ($^{\wedge}[0-9]+(\.[0-9]+)?\$$)	bad_price
date is not YYYY-MM-DD or not a real calendar date (month 1–12, day within the month, leap years)	bad_date

A trailing `\r` (Windows line endings) is removed first. Each distinct date string is validated once and the answer is cached, since the generator produces only about 730 distinct dates.

`./pipeline.sh transactions_malformed.tsv` (the supplied file: the first 10 sample rows plus 4 malformed rows). The pipeline does not stop, and the result is computed from the 10 valid rows:

category	transactions	revenue
Electronics	3	178000.00
Furniture	1	120000.00

The rejected rows are in `transactions_malformed.malformed_rows.tsv`:

bad_quantity	121	P21	Electronics	bad	10000	2026-06-25
bad_price	122	P22	Grocery	4		2026-06-26
bad_date	123	P23	Furniture	3	22000	invalid-date
field_count_5	124	P24	Clothing	2	5000	

Tested on generator output with `--malformed` (200,000 records, seed 7). 1,907 rows were rejected: 386 `bad_date` (2026-99-99), 382 `bad_price` (N/A), 390 `bad_quantity` (INVALID), 357 `field_count_4` and 392 `field_count_6` (extra field). The top-10 result matched an independent Python implementation exactly. Calendar edge cases were also tested: 2028-02-29 is accepted, while 2026-02-29, 2026-04-31 and quantity 3.5 are rejected.

The team's first version missed two of the generator's five malformed kinds. It accepted 2026-99-99 (format-only regex) and rows with an extra field (it checked only for *fewer* fields). Those rows were counted in the result and changed every category's count and revenue.

Scalability measurements

Same machine and configuration for every size: AMD Ryzen 7 5800H, 16 GB RAM, NVMe SSD, Windows 11, Git Bash (GNU awk 5.4.0, GNU sort 8.32), `LC_ALL=C`. Data from `generate_transactions --records N --seed 42`. Each size was run 3 times, after reading the file once so it is in the OS cache.

Records	Input size	Run 1	Run 2	Run 3	Median	s per MB
2,500,000	102.3 MB	10.85 s	10.65 s	10.93 s	10.9 s	0.106
6,250,000	255.8 MB	27.38 s	27.04 s	28.05 s	27.4 s	0.107
12,500,000	511.6 MB	64.21 s	58.93 s	54.97 s	58.9 s	0.115
25,000,000	1023.2 MB	109.45 s	107.86 s	109.37 s	109.4 s	0.107

Execution time grows linearly with input size. Doubling the input roughly doubles the time, and the cost stays at about 0.11 s per MB from 100 MB to 1 GB. The 500 MB runs varied most (55–64 s), probably because

other programs were running on the laptop at the same time.

Which stage dominates? (measured, `stage_timing.sh`)

`STOP_AFTER=<stage>` runs the pipeline only up to that stage, so the difference between successive rows is the time that stage adds. Medians of 3 runs:

Pipeline up to...	100 MB	250 MB	500 MB	1 GB
read (<code>tail</code>)	0.7 s	1.0 s	1.5 s	2.5 s
+ validate / filter / project (<code>awk</code>)	10.4 s	25.7 s	49.9 s	97.8 s
+ sort by category	10.6 s	26.6 s	50.5 s	108.5 s
+ aggregate / HAVING (<code>awk</code>)	11.1 s	27.7 s	53.4 s	105.3 s
full pipeline (+ order + limit)	10.9 s	27.5 s	55.9 s	110.4 s

At 1 GB, individual runs of the later prefixes vary by about ± 5 s (e.g. "+ sort": 111.4 / 101.4 / 108.5 s). So small differences between adjacent rows, including "+ sort" coming out above "+ aggregate", are measurement noise and not the cost of a stage.

- **The validation/filter `awk` takes about 90% of the time.** It is the only stage that processes every one of the millions of input rows with several checks (field count, two regular expressions, date check).
- **Reading the file is cheap** (0.7 s per 100 MB), so disk I/O is not the bottleneck.
- **sort adds under a second at 100–500 MB** (and at 1 GB less than the run-to-run noise). It only receives rows that passed the `WHERE` filter (about 45%: the generator's dates are spread evenly over 2025–2026, and 90% of quantities are above 2), each row is short, byte-wise comparison with `LC_ALL=C` is fast, and GNU sort uses several threads.
- **The aggregation, final sort and head add almost nothing.** Aggregation is a single cheap pass, and at most 20 category rows reach the final sort. Differences of a few tenths of a second between rows are within run-to-run noise.

Optimisation guided by this measurement. In the first version, the date check (regex, three `substr` calls, month-length arithmetic) ran on every row. Since there are only about 730 distinct dates, we cache the result per date string. Before vs after, same machine and data (`*_before_date_cache.tsv`):

Input	Before	After	Faster
102 MB	14.7 s	10.9 s	26%
256 MB	37.0 s	27.4 s	26%
512 MB	71.8 s	58.9 s	18%
1023 MB	149.9 s	109.4 s	27%

Both versions produce identical output.

Assumptions (stated explicitly)

- **Header row.** The input has a header row naming the columns (true for the supplied files and for `generate_transactions` output). Columns are located by name, because the supplied sample (`customer_id product_id category quantity price date`) does not use the column order written in the assignment (`transaction_id date category quantity price`).

- **Output rows.** "At most 10 rows" refers to data rows. The output also starts with one header line naming the required columns `category transactions revenue`.
- **Valid values.** Quantity must be a non-negative integer. Price must be a non-negative decimal number without sign or exponent (e.g. `12000`, `799.99`). The date must be a real calendar date in `YYYY-MM-DD` format. Anything else, including negative numbers, is treated as malformed.
- **Rows with extra fields** are treated as malformed, not truncated, since the schema has exactly as many fields as the header.
- **Generator arguments.** The assignment's example is `./generate_transactions 1000000`, but the supplied generator requires `--records N` (and accepts `--seed`). We used the supplied generator as it is: `--records N --seed 42`.
- **Revenue** is `quantity * price` summed in floating point and printed with 2 decimals.

What was attempted, achieved and not resolved

- **Achieved:** the full query as a streaming pipeline; malformed-row detection for all five malformed kinds the generator produces; measurements at ~100 MB, 250 MB, 500 MB and 1 GB (3 runs each); a measured per-stage breakdown; and one optimisation guided by that measurement.
- **Not attempted:** inputs larger than 1 GB (the assignment says no more than 1 GB is expected), and parallel splitting of the input.
- **Unresolved:** the 500 MB runs vary more (55–64 s) than the other sizes. We did not identify the cause.

Limitations of the Unix approach

- **Every run is a full scan.** There are no indexes, statistics or query optimiser. The order of the stages is fixed by hand, whereas a database could use an index on `date`, or choose a hash aggregate instead of sort-then-group.
- **Single-threaded bottleneck.** The dominant `awk` stage uses one core. We could split the file and run several `awk` processes in parallel, but that is manual work a database would do automatically.
- **Text-based types.** Validation uses regular expressions and string comparison (the date filter works only because `YYYY-MM-DD` sorts as text). Revenue is summed as floating point, so very large totals can be off by a few cents.
- **Portability.** Tested with GNU `awk` and GNU `coreutils`. The results committed with the team's first version were wrong because of the number formatting described above, which shows how much output format and `sort` flags matter.

Compliance checklist

Every requirement in the assignment PDF (*Nosql_Assignment1.pdf*), in the order it appears, with the file(s) that satisfy it. ✓ = met, ● = met with a stated limitation, ✗ = not met (the reason is given).

General instructions

#	Requirement (PDF)	Status	Where / how
G1	Team assignment; use the time to understand, analyse, implement and evaluate	✓	all three problems implemented, measured and analysed (see the three READMEs)
G2	For problems not fully solved, state what was attempted, achieved, unresolved and why	✓	<i>Limitations</i> sections in each README; P3 <i>What was attempted, achieved and not resolved</i> ; P1 <i>Limitations and honest notes</i>
G3	Submit all programs/scripts, datasets or dataset sources, commands, configurations and experimental outputs	✓	P1: all scripts, the official TPC-H kit, generated queries, raw timings, plans, outputs. P2: program, input CSVs, all outputs. P3: pipeline, supplied files, generator, measurements. Large generated datasets are replaced by their exact generation commands (<code>dbgen -s SF, generate_transactions --records N --seed 42</code>)
G4	Experiments reproducible; hardware/software environment and performance measurements documented	✓	P1 <code>results/environment.txt</code> , <code>results/tpch_provenance.txt</code> , setup script; P3 <i>Scalability measurements</i> (machine and tool versions); P2 README (Python version)
G5	Results from the team's own experiments; assumptions clearly stated	✓	all numbers come from the files in this package; <i>Assumptions</i> sections in P2 and P3; P1 <i>Limitations and honest notes</i>
G6	AI tools only as assistance; team responsible for correctness and understanding	—	responsibility of the team members; every result is backed by a script and raw output that can be re-run
G7	In-person demonstration; every member must understand every part	—	the READMEs explain the reasoning behind every design decision, as preparation for the viva

#	Requirement (PDF)	Status	Where / how
G8	Supplementary package used (customer CSVs, transactions TSVs, generator)	✓	byte-identical copies of the supplied files: P2 <code>customer_master.csv</code> , <code>customer_incoming.csv</code> ; P3 <code>transactions.tsv</code> , <code>transactions_malformed.tsv</code> , <code>generate_transactions</code> , <code>generate_transactions_README.txt</code>
G9	For TPC-H use the official TPC-H benchmark materials, not the supplied sample datasets	✓	official <i>TPC-H Tools v3.0.1</i> archive (unmodified), SHA-256 <code>97ccb34cd122d78c2e06e2419e50957f934256868b37c02d0b88aefd9d13a84a</code> , in <code>problem1_tpch/official_tpch_materials/</code> ; build and hashes in <code>problem1_tpch/results/tpch_provenance.txt</code>

Problem 1 — TPC-H scaling study

#	Requirement (PDF)	Status	Where / how
1.1	Use the current TPC-H specification and tools from TPC (Revision 3.0.1)	✓	official kit (G9). The tools print "Version 3.0.0 build 0", the banner of the programs inside the 3.0.1 kit, recorded exactly as observed
1.2	Increase the scale factor progressively, doubling each step	✓	SF 1, 2, 4, 8, 16, 32 (<code>results/totals.csv</code>)
1.3	For each SF, generate and load the dataset	✓	official <code>dbgen -s SF</code> , streamed COPY; generated = loaded rows for all 48 table/SF pairs (<code>results/row_counts.csv</code>)
1.4	Execute the specified TPC-H queries	✓	all 22, generated per SF by official <code>qgen -r 42 -s SF</code> (<code>queries/sf*/raw</code> , adapted <code>queries/sf*/</code> , SHA-256); SF1 results pass the official answer check 22/22, confirmed by the kit's <code>cmpq.pl</code> (<code>results/answer_validation*</code>)
1.5	Measure the execution time of each query and the total	✓	<code>results/timings.csv</code> (528 rows, all ok), <code>results/totals.csv</code>
1.6	Repeat the experiment as appropriate	✓	1 warm-up (excluded) + 3 measured runs per query per SF; medians, with min-max shown in the plots
1.7	Keep the experimental environment unchanged across scale factors	✓	same machine, PostgreSQL 16.15 build, configuration and 30 min timeout at every SF; both invocations recorded in <code>results/environment.txt</code> and identical except date/load/free space. The server is shared (load average 3.74 and 0.08 at the two starts), stated in the README limitations
1.8	State the database system (PostgreSQL preferred), its configuration, hardware and software environment	✓	README <i>Experimental setup</i> ; <code>results/environment.txt</code>
1.9	Plot dataset size against query execution time	✓	<code>results/scaling_plot.png</code> (total and per query vs dataset size), <code>results/per_query_plot.png</code> (22 panels)
1.10	Analyse whether time approximately doubles when the data doubles	✓	README <i>Does query time double...</i> : 33.0× data → 52.7× time; per-step ratios 1.72–2.67
1.11	Which queries scale linearly or super-linearly	✓	README sections 1 and 3, <code>results/scaling_summary.csv</code>
1.12	Which queries are relatively insensitive to dataset size	✓	README section 2: no query is insensitive overall; flat single steps (Q6, Q15, Q18) coincide with plan changes

#	Requirement (PDF)	Status	Where / how
1.13	At what SF performance begins to degrade significantly	✓	README section 3: SF32 (total ratio 2.67; seven queries > 3×), with plan evidence
1.14	Relate observations to joins, aggregation, sorting and filtering	✓	README <i>Relating the observations to database operations</i> , based on <code>results/plans/</code> (EXPLAIN at every SF, EXPLAIN ANALYZE at SF32)
1.15	Submit setup, dataset-generation procedure, execution scripts, raw measurements, plots, explanation	✓	<code>problem1_tpch/</code> (scripts, queries, <code>results/</code> , README)

Problem 2 — Incomplete data integration and JSON

#	Requirement (PDF)	Status	Where / how
2.1	Program that compares incoming with master and determines the relationship of every incoming record	✓	<code>match_customers.py</code> ; all 2000 rows in <code>classification_results.csv</code>
2.2	Each record in exactly one of the 5 categories	✓	one <code>match_status</code> per row; counts 400 + 657 + 193 + 400 + 350 = 2000 (<code>summary_statistics.txt</code>)
2.3	Own matching strategy, clearly described with rule order/priority and handling of missing and conflicting values	✓	README <i>Matching strategy</i> and <i>Classifying once a candidate is (or isn't) found</i>
2.4	Logically justified and consistently applied to all records	✓	README <i>Evidence behind the rule order</i> (measured uniqueness of each attribute); one code path for every row
2.5	Records with missing or conflicting attributes: do not discard, represent as JSON preserving available, missing and conflicting information	✓	<code>json_output/</code> (1202 documents) with <code>available_information</code> , <code>missing_information</code> , <code>conflicting_information</code>
2.6	JSON structure may differ from the example but must explicitly identify missing/conflicting information	✓	explicit <code>missing_information</code> list and <code>conflicting_information</code> object (incoming vs master value)
2.7	Submit algorithm and source code	✓	<code>match_customers.py</code> , README
2.8	Submit matching criteria with justification	✓	README <i>Matching strategy</i> , <i>Evidence behind the rule order</i>
2.9	Submit the classification of all records	✓	<code>classification_results.csv</code>
2.10	JSON documents for records with missing, irregular or conflicting information	✓	all 1200 partial/incomplete/conflicting records, plus 2 records whose only issue is an irregular value; 41 documents carry <code>irregular_information</code> with the raw value (README <i>Irregular values</i>)
2.11	Brief explanation of why JSON suits such records	✓	README <i>Why JSON for records with missing, conflicting or irregular information</i>
2.12	Summary statistics for each category	✓	<code>summary_statistics.txt</code> ; README <i>Results on the supplied data</i>
2.13	Discuss assumptions, limitations and difficulties	✓	README <i>Assumptions / limitations</i> , <i>Difficulties encountered</i>
2.14	Supplied CSV files used as input	✓	the program reads <code>customer_master.csv</code> and <code>customer_incoming.csv</code> (byte-identical to the supplied files)

#	Requirement (PDF)	Status	Where / how
2.15	Do not assume the example JSON records exist	✓	no hard-coded records; ids such as <code>C00104</code> are not referenced
2.16	All records processed and classified automatically, none manually	✓	single loop over all rows; no per-record special cases in the code
2.17	Matching rules consistent across the dataset	✓	same <code>classify()</code> for every row

Problem 3 — Unix data-processing pipeline

#	Requirement (PDF)	Status	Where / how
3.1	SQL-like task using only Unix command-line tools and shell scripting	✓	<code>pipeline.sh</code> : <code>head</code> , <code>tail</code> , <code>tr</code> , <code>grep</code> , <code>cut</code> , <code>awk</code> , <code>sort</code> in a bash script
3.2	Process the TSV without loading the whole file into memory	✓	every stage streams line by line; <code>sort</code> spills to temp files beyond 256 MB; aggregation keeps one group's state at a time (README <i>Memory use</i>)
3.3	Fields <code>transaction_id</code> <code>date</code> <code>category</code> <code>quantity</code> <code>price</code> , tab-separated, typed as specified	✓	columns located by header name; types validated (README <i>Malformed-record handling, Assumptions</i>)
3.4	Implement the given <code>SELECT ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT 10</code>	✓	README <i>Processing stages and SQL → Unix mapping</i> ; results match an independent Python calculation
3.5	Do not import into a database or run the SQL directly	✓	no database or SQL engine is used
3.6	Multiple stages connected by pipes, reading the input progressively	✓	<code>tail \ awk \ sort \ awk \ sort \ head</code>
3.7	Selection, projection, filtering, grouping, aggregation, sorting and Top-K	✓	mapping table in the README
3.8	Demonstrate malformed-record handling (missing field, invalid date, non-numeric quantity or price); exclude them without terminating	✓	README <i>Malformed-record handling</i> : supplied malformed file (4 rows rejected with reasons) and a generated <code>--malformed</code> file (1,907 rows, all 5 kinds)
3.9	Use the supplied generator; it takes the number of records	⦿	used unchanged as <code>generate_transactions --records N --seed 42</code> . The supplied generator requires <code>--records</code> , unlike the PDF's positional example (README <i>Assumptions</i>)
3.10	Develop and test on the supplied sample first	✓	README <i>Sample input and output</i>
3.11	At least four sizes $\approx$ 100 MB, 250 MB, 500 MB, 1 GB	✓	102.3 / 255.8 / 511.6 / 1023.2 MB (<code>scalability_results.tsv</code>)
3.12	Record input size and total execution time; same machine and configuration	✓	<code>scalability_results.tsv</code> (3 runs each), machine stated in the README
3.13	Report a table; discuss how time changes with size	✓	README <i>Scalability measurements</i>
3.14	Explain which stages likely dominate and why	✓	measured per stage (<code>stage_timing.sh</code> , <code>stage_timings.tsv</code>), README <i>Which stage dominates?</i>

#	Requirement (PDF)	Status	Where / how
3.15	Output at most 10 rows with columns <code>category transactions revenue</code> , ordered by decreasing revenue	✓	<code>outputs/out_*.tsv</code> ; header line plus ≤ 10 data rows (README <i>Assumptions</i>)
3.16	Submit the complete shell script	✓	<code>pipeline.sh</code>
3.17	Sample input and output	✓	README <i>Sample input and output</i>
3.18	Explanation of the processing stages	✓	README <i>Processing stages</i>
3.19	The equivalent SQL query	✓	README <i>Equivalent SQL</i>
3.20	Mapping of each SQL operation to Unix commands	✓	README mapping table
3.21	Scalability measurements and analysis of performance and limitations	✓	README <i>Scalability measurements, Limitations of the Unix approach</i>
3.22	Explain how malformed records are detected and handled	✓	README <i>Malformed-record handling</i>
3.23	Operate directly on TSV files; no database, dataframe library or in-memory framework	✓	only <code>awk/sort/coreutils</code>
3.24	Intermediate files only where necessary; use pipes and streaming	✓	the only files written besides the output are the malformed-rows log and <code>sort</code> 's own temporary files
3.25	Work with generator files; no fixed record count or input filename	✓	input path is an argument; tested from the 14-row supplied malformed file up to 25,000,000 generated rows, under several different file names